

Polymorphism means "many forms"—it allows different objects to respond to the same method call in their own unique ways. In Python, polymorphism is driven primarily by **Duck Typing**, a philosophy where an object's identity is defined by what methods and properties it has, rather than what class it inherits from.

1. Built-in Polymorphism

Python uses polymorphism everywhere out of the box. For instance, the `len()` function doesn't care what type of object you give it, as long as that object knows how to report its length.

```
# The same function len() handles completely different object types
print(len("Hello World"))      # String -> 11
print(len([10, 20, 30, 40]))   # List    -> 4
print(len({"a": 1, "b": 2}))    # Dict    -> 2
```

2. Polymorphism via Inheritance (Classic OOP)

In traditional Object-Oriented Programming, polymorphism is achieved when subclasses override a method defined in a parent class.

```
class PaymentProcessor:
    def process_payment(self, amount: float):
        raise NotImplementedError("Subclasses must implement this method")

class CreditCardProcessor(PaymentProcessor):
    def process_payment(self, amount: float):
        print(f"Processing ${amount:.2f} via Credit Card.")

class PayPalProcessor(PaymentProcessor):
    def process_payment(self, amount: float):
        print(f"Processing ${amount:.2f} via PayPal.")

# A single function that handles any PaymentProcessor child class
def checkout(processor: PaymentProcessor, amount: float):
    processor.process_payment(amount)

checkout(CreditCardProcessor(), 49.99)
checkout(PayPalProcessor(), 15.00)
```

3. Duck Typing ("If it walks like a duck...")

The phrase comes from the adage: *"If it walks like a duck and quacks like a duck, it's a duck."*

In languages like Java or C++, `checkout()` would **require** the object to explicitly inherit from `PaymentProcessor`. Python does **not** check the class hierarchy. It only checks if the object currently has the method being called at runtime.

```
# CryptoProcessor does NOT inherit from PaymentProcessor!
class CryptoProcessor:
    def process_payment(self, amount: float):
        print(f"Processing ${amount:.2f} via Bitcoin transfer.")

# Duck typing in action: checkout() still works because CryptoProcessor has process_
payment()
checkout(CryptoProcessor(), 120.00)
```

Hands-On Custom Duck Typing Example

You can implement duck typing in your own classes to integrate seamlessly with Python's built-in functions.

For example, to make a custom class work with Python's built-in `len()` function, you simply implement the `__len__` method (often called a "dunder" or magic method):

```
class BookClub:
    def __init__(self, members: list[str]):
        self.members = members

    # By defining __len__, BookClub now "quacks" like a sequence/container
    def __len__(self):
        return len(self.members)

club = BookClub(["Alice", "Bob", "Charlie"])

# len() works immediately without BookClub inheriting from list or any special class
print(f"Total members: {len(club)}") # Output: Total members: 3
```

Key Summary

Concept	How it Works	Key Difference
Polymorphism	Different objects execute different code through the same interface or method name.	The concept of unified method signatures.
Duck Typing	Python checks for method existence at runtime rather than matching class inheritance.	The mechanism Python uses to achieve flexible polymorphism.

What you are describing is specifically **Method Overloading**, which is just **one specific type** of polymorphism common in statically typed languages like Java and C++.

Broadly, **Polymorphism** ("many forms") means giving a single interface to entities of different types. It splits into two main categories:

1. Method Overloading (Compile-Time / Static Polymorphism)

- **What it is:** Defining multiple methods in the **same class** with the **same name**, but **different parameter signatures** (different number or types of arguments).
- **Does Python support this natively? NO.**

In Python, if you define two methods with the exact same name in the same class, the second definition **overwrites** the first one:

```
# ❌ How it works in Python (Python DOES NOT support native overloading):
class Calculator:
    def add(self, a, b):
        return a + b

    def add(self, a, b, c): # This line OVERWRITES the previous add() method!
        return a + b + c

calc = Calculator()
# calc.add(2, 3)      <-- CRASHES! TypeError: missing 1 required positional argument:
# 'c'
print(calc.add(2, 3, 4)) # Output: 9
```

How Python Handles "Overloading"

Because Python is dynamically typed, you achieve varying method behavior in a single method using **default arguments** or **varargs** (`*args`):

```
# ✅ The Pythonic way to handle varying parameter lists
class Calculator:
    def add(self, *args):
        return sum(args)

calc = Calculator()
print(calc.add(2, 3))          # Output: 5
print(calc.add(2, 3, 4, 5))    # Output: 14
```

2. Method Overriding (Runtime / Dynamic Polymorphism)

- **What it is:** Defining the **same method signature** across **different classes** (usually through inheritance or duck typing) so each class provides its own specific implementation.
- **Does Python support this? YES.** This is what Python relies on heavily.

```
class Dog:
    def speak(self):
        return "Woof!"

class Cat:
    def speak(self):
        return "Meow!"

# The same method call .speak() behaves differently based on the object's class
for animal in [Dog(), Cat()]:
    print(animal.speak())
```

Direct Comparison

Feature	Method Overloading	Method Overriding
Where it happens	Same class	Different classes
Method Signature	Different parameters/types	Same method name and parameters
Type of Polymorphism	Static (Compile-time)	Dynamic (Runtime)
Native Python Support?	 No (Simulated via <code>*args</code> or <code>@singledispatch</code>)	 Yes (Native via Duck Typing & Classes)